

Python Q1 (OOPs) — Supply Chain Inventory Tracker

ANSWER

Build a small in-memory inventory registry that adds stock, fulfills orders, processes returned stock, and lists products that still have stock.

Create Class and State

```
class SupplyChainInventory:
    def __init__(self):
        self.inv = {}
```

Step 1 — Add Product Stock

Implementation Flow:

- If product_id exists, increase its quantity.
- Otherwise create a new entry.
- Return the dictionary.

Function:

```
def add_product(self, product_id, quantity):
    if product_id in self.inv:
        self.inv[product_id] += quantity
    else:
        self.inv[product_id] = quantity
    result = self.inv
    return result
```

Step 2 — Fulfill Order

Implementation Flow:

- Check whether product exists and has enough stock.
- If not, raise ValueError("Insufficient Stock").
- Subtract quantity.
- Return the dictionary.

Function:

```
def fulfill_order(self, product_id, quantity):
    if product_id not in self.inv or self.inv[product_id] < quantity:
        raise ValueError("Insufficient Stock")
    self.inv[product_id] -= quantity
    result = self.inv
    return result
```

Step 3 — Restock Return

Implementation Flow:

- If product exists, increase quantity.
- Otherwise create a new entry.
- Return the dictionary.

Function:

```
def restock_return(self, product_id, quantity):
    if product_id in self.inv:
        self.inv[product_id] += quantity
    else:
        self.inv[product_id] = quantity
    result = self.inv
    return result
```

Step 4 — List Available Stock

Implementation Flow:

- Loop through inventory.
- Keep product IDs whose stock is greater than 0.
- Return the list.

Function:

```
def list_available_products(self):
    result = []
    for product_id in self.inv:
        if self.inv[product_id] > 0:
            result.append(product_id)
    return result
```

Complete Code

```
class SupplyChainInventory:
    def __init__(self):
        self.inv = {}

    def add_product(self, product_id, quantity):
        if product_id in self.inv:
            self.inv[product_id] += quantity
        else:
            self.inv[product_id] = quantity
        result = self.inv
        return result

    def fulfill_order(self, product_id, quantity):
        if product_id not in self.inv or self.inv[product_id] < quantity:
            raise ValueError("Insufficient Stock")
        self.inv[product_id] -= quantity
        result = self.inv
        return result

    def restock_return(self, product_id, quantity):
        if product_id in self.inv:
            self.inv[product_id] += quantity
        else:
            self.inv[product_id] = quantity
        result = self.inv
        return result

    def list_available_products(self):
        result = []
        for product_id in self.inv:
            if self.inv[product_id] > 0:
                result.append(product_id)
        return result
```

Easy Memory Order

- Add → check ID → increase/create → return
- Fulfill → check stock → subtract → return
- Return → increase/create → return
- List → stock > 0

Python Q2 (OOPs) — Event Registration Tracker

ANSWER

Build an event registration system that registers participants, cancels registrations, reschedules an event, and lists active events.

Create Class and State

```
class EventRegistrationTracker:
    def __init__(self):
        self.events = {}
```

Step 1 — Register Participant

Implementation Flow:

- If event exists, increase count.
- Otherwise create event.
- Return dictionary.

Function:

```
def register_participant(self, event_id, count):
    if event_id in self.events:
        self.events[event_id] += count
    else:
        self.events[event_id] = count
    result = self.events
    return result
```

Step 2 — Cancel Participant

Implementation Flow:

- Check event exists and enough participants are registered.
- Otherwise raise ValueError("Cannot cancel more participants than registered").
- Subtract count.
- Return dictionary.

Function:

```
def cancel_participant(self, event_id, count):
    if event_id not in self.events or self.events[event_id] < count:
        raise ValueError("Cannot cancel more participants than registered")
    self.events[event_id] -= count
    result = self.events
    return result
```

Step 3 — Reschedule Event

Implementation Flow:

- If old event is absent, return current dictionary.
- Take old count.
- Add count to new event or create it.
- Delete old event.
- Return dictionary.

Function:

```
def reschedule_event(self, old_event_id, new_event_id):
    if old_event_id not in self.events:
        return self.events
    count = self.events[old_event_id]
    if new_event_id in self.events:
        self.events[new_event_id] += count
```

```

else:
    self.events[new_event_id] = count
del self.events[old_event_id]
result = self.events
return result

```

Step 4 — Get Active Events

Implementation Flow:

- Loop through events.
- Keep IDs where participant count > 0.
- Return list.

Function:

```

def get_active_events(self):
    result = []
    for event_id in self.events:
        if self.events[event_id] > 0:
            result.append(event_id)
    return result

```

Complete Code

```

class EventRegistrationTracker:
    def __init__(self):
        self.events = {}

    def register_participant(self, event_id, count):
        if event_id in self.events:
            self.events[event_id] += count
        else:
            self.events[event_id] = count
        result = self.events
        return result

    def cancel_participant(self, event_id, count):
        if event_id not in self.events or self.events[event_id] < count:
            raise ValueError("Cannot cancel more participants than registered")
        self.events[event_id] -= count
        result = self.events
        return result

    def reschedule_event(self, old_event_id, new_event_id):
        if old_event_id not in self.events:
            return self.events
        count = self.events[old_event_id]
        if new_event_id in self.events:
            self.events[new_event_id] += count
        else:
            self.events[new_event_id] = count
        del self.events[old_event_id]
        result = self.events
        return result

    def get_active_events(self):
        result = []
        for event_id in self.events:
            if self.events[event_id] > 0:
                result.append(event_id)
        return result

```

Python Q3 (OOPs) — Smart City Traffic Control System

ANSWER

Build a traffic control system that stores vehicle counts at intersections, updates counts, finds congested intersections, and assigns signal timing.

Create Class and State

```
class TrafficControlSystem:
    def __init__(self):
        self.traffic_data = {}
```

Step 1 — Add a New Intersection

Implementation Flow:

- Store intersection as key and vehicle count as value.
- Return dictionary.

Function:

```
def add_intersection(self, intersection, vehicle_count):
    self.traffic_data[intersection] = vehicle_count
    result = self.traffic_data
    return result
```

Step 2 — Update Vehicle Count

Implementation Flow:

- If intersection is missing, return "Error: Intersection not found".
- Otherwise update the count.
- Return dictionary.

Function:

```
def update_vehicle_count(self, intersection, new_count):
    if intersection not in self.traffic_data:
        return "Error: Intersection not found"
    self.traffic_data[intersection] = new_count
    result = self.traffic_data
    return result
```

Step 3 — Identify Congested Intersections

Implementation Flow:

- Loop through intersections.
- Keep entries where count > threshold.
- Return dictionary.

Function:

```
def get_congested_intersections(self, congestion_threshold):
    result = {}
    for intersection in self.traffic_data:
        if self.traffic_data[intersection] > congestion_threshold:
            result[intersection] = self.traffic_data[intersection]
    return result
```

Step 4 — Adjust Traffic Signals

Implementation Flow:

- `count > 80` → "Long Green".
- `40–80` → "Normal Green".
- Otherwise → "Short Green".
- Return signal dictionary.

Function:

```
def adjust_traffic_signals(self):
    result = {}
    for intersection in self.traffic_data:
        count = self.traffic_data[intersection]
        if count > 80:
            result[intersection] = "Long Green"
        elif count >= 40:
            result[intersection] = "Normal Green"
        else:
            result[intersection] = "Short Green"
    return result
```

Complete Code

```
class TrafficControlSystem:
    def __init__(self):
        self.traffic_data = {}

    def add_intersection(self, intersection, vehicle_count):
        self.traffic_data[intersection] = vehicle_count
        result = self.traffic_data
        return result

    def update_vehicle_count(self, intersection, new_count):
        if intersection not in self.traffic_data:
            return "Error: Intersection not found"
        self.traffic_data[intersection] = new_count
        result = self.traffic_data
        return result

    def get_congested_intersections(self, congestion_threshold):
        result = {}
        for intersection in self.traffic_data:
            if self.traffic_data[intersection] > congestion_threshold:
                result[intersection] = self.traffic_data[intersection]
        return result

    def adjust_traffic_signals(self):
        result = {}
        for intersection in self.traffic_data:
            count = self.traffic_data[intersection]
            if count > 80:
                result[intersection] = "Long Green"
            elif count >= 40:
                result[intersection] = "Normal Green"
            else:
                result[intersection] = "Short Green"
```

Python Q4 (OOPs) — Chess Tournament Player Mang. System

ANSWER

Build class-based player management system that adds players, updates ratings, returns player details, and lists qualified players.

Create Class and State

```
class ChessTournamentSystem:
    def __init__(self):
        self.players = {}
```

Step 1 — Add Player

Implementation Flow:

- If player_id already exists, raise ValueError.
- Store name, rating, and status = Active.
- Return dictionary.

Function:

```
def add_player(self, player_id, name, rating):
    if player_id in self.players:
        raise ValueError("Player already exists")
    self.players[player_id] = {
        "name": name,
        "rating": rating,
        "status": "Active"
    }
    result = self.players
    return result
```

Step 2 — Update Rating

Implementation Flow:

- Check player exists.
- If missing, raise KeyError.
- Update rating.
- Return dictionary.

Function:

```
def update_rating(self, player_id, new_rating):
    if player_id not in self.players:
        raise KeyError("Player not found")
    self.players[player_id]["rating"] = new_rating
    result = self.players
    return result
```

Step 3 — Get Player Details

Implementation Flow:

- Check player exists.
- If missing, raise KeyError.
- Return stored details.

Function:

```
def get_player_details(self, player_id):
    if player_id not in self.players:
        raise KeyError("Player not found")
    result = self.players[player_id]
    return result
```

Step 4 — List Qualified Players

Implementation Flow:

- Loop through players.
- Select rating $\geq$ minimum_rating.
- Return player IDs.

Function:

```
def qualified_players(self, minimum_rating):
    result = []
    for player_id in self.players:
        if self.players[player_id]["rating"] >= minimum_rating:
            result.append(player_id)
    return result
```

Complete Code

```
class ChessTournamentSystem:
    def __init__(self):
        self.players = {}

    def add_player(self, player_id, name, rating):
        if player_id in self.players:
            raise ValueError("Player already exists")
        self.players[player_id] = {
            "name": name,
            "rating": rating,
            "status": "Active"
        }
        result = self.players
        return result

    def update_rating(self, player_id, new_rating):
        if player_id not in self.players:
            raise KeyError("Player not found")
        self.players[player_id]["rating"] = new_rating
        result = self.players
        return result

    def get_player_details(self, player_id):
        if player_id not in self.players:
            raise KeyError("Player not found")
        result = self.players[player_id]
        return result

    def qualified_players(self, minimum_rating):
        result = []
        for player_id in self.players:
            if self.players[player_id]["rating"] >= minimum_rating:
                result.append(player_id)
        return result
```

Easy Memory Order

- Add → duplicate check → store
- Update → check ID → change rating
- Get → check ID → return details
- Qualified → rating $\geq$ minimum

Python Q5 (Pandas) — Insurance Claim Processing

ANSWER

Build a ClaimAnalyzer class that creates a claims DataFrame, calculates approval rate by category, flags high-value claims, finds top pending claims, and summarizes amounts by status.

Create Class and State

```
class ClaimAnalyzer:
```

Step 1 — Create Claims DataFrame

Implementation Flow:

- Create DataFrame with CustomerID, Category, Amount, Status, Date.
- Return DataFrame.

Function:

```
def create_claims_df(self, claim_data):  
    df = pd.DataFrame(claim_data,  
        columns=["CustomerID", "Category", "Amount", "Status", "Date"])  
    return df
```

Step 2 — Approval Rate by Category

Implementation Flow:

- Group by Category for total count.
- Filter Approved and count by Category.
- Calculate approved / total * 100.
- Reset index and name columns.

Function:

```
def approval_rate_by_category(self, df):  
    total = df.groupby("Category").size()  
    approved = df[df["Status"] == "Approved"].groupby("Category").size()  
    rate = (approved / total * 100).fillna(0)  
    result = rate.reset_index()  
    result.columns = ["Category", "Approval Rate"]  
    return result
```

Step 3 — Add High Amount Flag

Implementation Flow:

- Add IsHighValue.
- Use Amount > threshold.
- Return DataFrame.

Function:

```
def add_flag_high_amount(self, df, threshold):  
    df = df.copy()  
    df["IsHighValue"] = df["Amount"] > threshold  
    return df
```

Step 4 — Get Top Pending Claims

Implementation Flow:

- Filter Status == Pending.
- Sort Amount descending.
- Take top n rows.

Function:

```
def get_top_pending_claims(self, df, n):
    result = df[df["Status"] == "Pending"]
    result = result.sort_values("Amount", ascending=False)
    result = result.head(n)
    return result
```

Step 5 — Claims Summary by Status

Implementation Flow:

- Group by Status.
- Aggregate sum, min, max, mean.
- Reset index and rename mean to avg.

Function:

```
def claim_summary_by_status(self, df):
    result = df.groupby("Status")["Amount"].agg(
        ["sum", "min", "max", "mean"]
    ).reset_index()
    result.columns = ["Status", "sum", "min", "max", "avg"]
    return result
```

Complete Code

```
import pandas as pd

class ClaimAnalyzer:
    def create_claims_df(self, claim_data):
        df = pd.DataFrame(claim_data,
            columns=["CustomerID", "Category", "Amount", "Status", "Date"])
        return df

    def approval_rate_by_category(self, df):
        total = df.groupby("Category").size()
        approved = df[df["Status"] == "Approved"].groupby("Category").size()
        rate = (approved / total * 100).fillna(0)
        result = rate.reset_index()
        result.columns = ["Category", "Approval Rate"]
        return result

    def add_flag_high_amount(self, df, threshold):
        df["IsHighValue"] = df["Amount"] > threshold
        return df

    def get_top_pending_claims(self, df, n):
        result = df[df["Status"] == "Pending"]
        result = result.sort_values("Amount", ascending=False)
        result = result.head(n)
        return result

    def claim_summary_by_status(self, df):
        result = df.groupby("Status")["Amount"].agg(["sum", "min", "max", "mean"]).reset_index()
        result.columns = ["Status", "sum", "min", "max", "avg"]
        return result
```

Python Q6 (Pandas) — Solar Farm Production Analyzer

Create Class and State

```
class SolarFarmAnalyzer:
```

Step 1 — Create Production DataFrame

Implementation Flow:

- Create DataFrame with TurbineID, Date, Energy, WindSpeed, OutageMinutes.
- Return DataFrame.

Function:

```
def create_production_df(self, data):  
    df = pd.DataFrame(data,  
        columns=["TurbineID", "Date", "Energy", "WindSpeed", "OutageMinutes"])  
    return df
```

Step 2 — Total Energy per Turbine

Implementation Flow:

- Group by TurbineID.
- Sum Energy.
- Rename to TotalEnergy.
- Reset index.

Function:

```
def total_energy_per_turbine(self, df):  
    result = df.groupby("TurbineID")["Energy"].sum().reset_index()  
    result = result.rename(columns={"Energy": "TotalEnergy"})  
    return result
```

Step 3 — Add Energy per Uptime Minute

Implementation Flow:

- Assume 1440 minutes per day.
- ActiveMinutes = 1440 - OutageMinutes.
- EnergyPerMin = Energy / ActiveMinutes.
- Return updated DataFrame.

Function:

```
def add_energy_per_min(self, df):  
    df = df.copy()  
    active_minutes = 1440 - df["OutageMinutes"]  
    df["EnergyPerMin"] = df["Energy"] / active_minutes  
    return df
```

Step 4 — Categorize Wind Band

Implementation Flow:

- Start WindBand as "Low".
- WindSpeed >= 3 → Moderate.
- WindSpeed >= 7 → High.
- Return DataFrame.

Function:

```
def categorize_wind_band(self, df):
    df = df.copy()
    df["WindBand"] = "Low"
    df.loc[df["WindSpeed"] >= 3, "WindBand"] = "Moderate"
    df.loc[df["WindSpeed"] >= 7, "WindBand"] = "High"
    return df
```

Step 5 — Frequent Outage Rows

Implementation Flow:

- Filter OutageMinutes > n.
- Return DataFrame.

Function:

```
def frequent_outage_rows(self, df, n):
    result = df[df["OutageMinutes"] > n]
    return result
```

Step 6 — Clean & Top Production Days

Implementation Flow:

- Drop rows containing NaN.
- Sort Energy descending.
- Return clean sorted DataFrame.

Function:

```
def clean_and_top_days(self, df):
    result = df.dropna()
    result = result.sort_values("Energy", ascending=False)
    return result
```

Complete Code

```
import pandas as pd
class SolarFarmAnalyzer:
    def create_production_df(self, data):
        df = pd.DataFrame(data,
                           columns=["TurbineID", "Date", "Energy", "WindSpeed", "OutageMinutes"])
        return df
    def total_energy_per_turbine(self, df):
        result = df.groupby("TurbineID")["Energy"].sum().reset_index()
        result = result.rename(columns={"Energy": "TotalEnergy"})
        return result
    def add_energy_per_min(self, df):
        active_minutes = 1440 - df["OutageMinutes"]
        df["EnergyPerMin"] = df["Energy"] / active_minutes
        return df
    def categorize_wind_band(self, df):
        df["WindBand"] = "Low"
        df.loc[df["WindSpeed"] >= 3, "WindBand"] = "Moderate"
        df.loc[df["WindSpeed"] >= 7, "WindBand"] = "High"
        return df
    def frequent_outage_rows(self, df, n):
        result = df[df["OutageMinutes"] > n]
        return result
    def clean_and_top_days(self, df):
        result = df.dropna()
        result = result.sort_values("Energy", ascending=False)
        return result
```

Python Q7 (Pandas) — Marketplace Returns and Refund Analysis

ANSWER

Build a ReturnsAnalyzer class to create order/return DataFrames, merge them, calculate category refund rates, identify high-return sellers, and clean invalid return records.

Create Class and State

```
class ReturnsAnalyzer:
```

Step 1 — Create Orders DataFrame

Implementation Flow:

- Create DataFrame with OrderID, SellerID, Category, OrderDate, OrderAmount.
- Return it.

Function:

```
def create_orders_df(self, order_data):
    df = pd.DataFrame(order_data,
                       columns=["OrderID", "SellerID", "Category", "OrderDate", "OrderAmount"])
    return df
```

Step 2 — Create Returns DataFrame

Implementation Flow:

- Create DataFrame with OrderID, ReturnDate, RefundAmount, Reason.
- Return it.

Function:

```
def create_returns_df(self, return_data):
    df = pd.DataFrame(
        return_data,
        columns=["OrderID", "ReturnDate", "RefundAmount", "Reason"]
    )
    return df
```

Step 3 — Merge Orders with Returns

Implementation Flow:

- Merge on OrderID.
- Use a left merge so all orders remain.
- Return merged DataFrame.

Function:

```
def merge_orders_returns(self, orders_df, returns_df):
    result = pd.merge(orders_df, returns_df, on="OrderID", how="left")
    return result
```

Step 4 — Category-wise Refund Rate

Implementation Flow:

- Create IsReturned from non-null RefundAmount.
- Group by Category.
- Count orders and returned orders.
- Calculate ReturnedOrders / Orders * 100.
- Sort by Category and reset index.

Function:

```
def category_refund_rate(self, merged_df):
    df = merged_df.copy()
    df["IsReturned"] = df["RefundAmount"].notna().astype(int)

    orders = df.groupby("Category").size()
    returned = df.groupby("Category")["IsReturned"].sum()

    rate = returned / orders * 100
    result = rate.reset_index()
    result.columns = ["Category", "RefundRate"]
    return result
```

Step 5 — High Return Sellers

Implementation Flow:

- Keep IsReturned == 1.
- Group by SellerID and count.
- Keep ReturnCount > n.
- Reset index.

Function:

```
def high_return_sellers(self, merged_df, n):
    result = merged_df[merged_df["IsReturned"] == 1]
    result = result.groupby("SellerID").size()
    result = result.reset_index(name="ReturnCount")
    result = result[result["ReturnCount"] > n]
    result = result.reset_index(drop=True)
    return result
```

Step 6 — Clean Return Records

Implementation Flow:

- Drop rows with null values.
- Keep RefundAmount > 0.
- Reset index.
- Return cleaned DataFrame.

Function:

```
def clean_returns_data(self, returns_df):
    result = returns_df.dropna()
    result = result[result["RefundAmount"] > 0]
    result = result.reset_index(drop=True)
    return result
```

Complete Code

```
import pandas as pd

class ReturnsAnalyzer:
    def create_orders_df(self, order_data):
        df = pd.DataFrame(
            order_data,
            columns=["OrderID", "SellerID", "Category", "OrderDate", "OrderAmount"]
        )
        return df

    def create_returns_df(self, return_data):
        df = pd.DataFrame(
            return_data,
            columns=["OrderID", "ReturnDate", "RefundAmount", "Reason"]
        )
        return df

    def merge_orders_returns(self, orders_df, returns_df):
        result = pd.merge(orders_df, returns_df, on="OrderID", how="left")
        return result

    def category_refund_rate(self, merged_df):
        df = merged_df.copy()
        df["IsReturned"] = df["RefundAmount"].notna().astype(int)

        orders = df.groupby("Category").size()
        returned = df.groupby("Category")["IsReturned"].sum()
        rate = returned / orders * 100
        result = rate.reset_index()
        result.columns = ["Category", "RefundRate"]
        return result

    def high_return_sellers(self, merged_df, n):
        result = merged_df[merged_df["IsReturned"] == 1]
        result = result.groupby("SellerID").size()
        result = result.reset_index(name="ReturnCount")
        result = result[result["ReturnCount"] > n]
        result = result.reset_index(drop=True)
        return result

    def clean_returns_data(self, returns_df):
        result = returns_df.dropna()
        result = result[result["RefundAmount"] > 0]
        result = result.reset_index(drop=True)
        return result
```

Python Q8 (Pandas) — Electric Vehicle Charging Station Analysis

ANSWER

Build an EVChargingAnalyzer class using Pandas. The solution follows the six required operations and keeps ChargingDate as a string as specified.

Class Declaration

```
class EVChargingAnalyzer:
```

1. Create Charging Sessions DataFrame

Implementation Flow:

- Create a DataFrame using the seven required columns.
- Keep ChargingDate as a string.
- Return the DataFrame.

Function:

```
def create_sessions_df(self, session_data):
    df = pd.DataFrame(
        session_data,
        columns=[
            "SessionID", "StationID", "City", "ChargingDate",
            "EnergyKWh", "DurationMinutes", "PaymentStatus"
        ]
    )
    return df
```

2. Clean Charging Sessions

Implementation Flow:

- Drop rows where SessionID, StationID, City, or PaymentStatus is null.
- Keep only EnergyKWh > 0 and DurationMinutes > 0.
- Keep PaymentStatus only in "Paid", "Pending", and "Failed".
- Return the cleaned DataFrame.

Function:

```
def clean_sessions_data(self, df):
    result = df.dropna(
        subset=["SessionID", "StationID", "City", "PaymentStatus"]
    )
    result = result[(result["EnergyKWh"] > 0) & (result["DurationMinutes"] > 0)]
    result = result[result["PaymentStatus"].isin(["Paid", "Pending", "Failed"])]
    return result
```

3. Add Long Session Flag

Implementation Flow:

- Create IsLongSession.
- Set it to 1 when DurationMinutes > duration_threshold.
- Otherwise set it to 0.
- Return the updated DataFrame.

Function:

```
def add_long_session_flag(self, df, duration_threshold):
    result = df.copy()
    result["IsLongSession"] = (result["DurationMinutes"] > duration_threshold).astype(int)
    return result
```

4. Station Utilization Summary

Implementation Flow:

- Group by StationID.
- Count sessions.
- Sum EnergyKWh.
- Calculate mean DurationMinutes.
- Round AverageDuration to 1 decimal.
- Reset index and return.

Function:

```
def station_utilization_summary(self, df):
    result = df.groupby("StationID").agg(
        SessionCount=("SessionID", "count"),
        TotalEnergyKWh=("EnergyKWh", "sum"),
        AverageDuration=("DurationMinutes", "mean")
    ).reset_index()

    result["AverageDuration"] = result["AverageDuration"].round(1)
    return result
```

5. High Energy Stations

Implementation Flow:

- Group by StationID.
- Sum EnergyKWh.
- Rename the output column TotalEnergyKWh.
- Keep only rows where TotalEnergyKWh > energy_threshold.
- Use strict greater-than comparison.
- Return the result.

Function:

```
def high_energy_stations(self, df, energy_threshold):
    result = df.groupby("StationID")["EnergyKWh"].sum().reset_index()
    result = result.rename(columns={"EnergyKWh": "TotalEnergyKWh"})
    result = result[result["TotalEnergyKWh"] > energy_threshold]
    return result
```

6. City Revenue Summary

Implementation Flow:

- Filter PaymentStatus == Paid.
- Create Revenue = EnergyKWh * 18.
- Group by City and sum Revenue.
- Rename the output column Revenue.
- Reset index and return.

Function:

```
def city_revenue_summary(self, df):
    result = df[df["PaymentStatus"] == "Paid"].copy()
    result["Revenue"] = result["EnergyKWh"] * 18
    result = result.groupby("City")["Revenue"].sum().reset_index()
    return result
```

Complete Code

```
import pandas as pd

class EVChargingAnalyzer:

    def create_sessions_df(self, session_data):
        df = pd.DataFrame(
            session_data,
            columns=[
                "SessionID", "StationID", "City", "ChargingDate",
                "EnergyKWh", "DurationMinutes", "PaymentStatus"
            ]
        )
        return df

    def clean_sessions_data(self, df):
        result = df.dropna(
            subset=["SessionID", "StationID", "City", "PaymentStatus"]
        )
        result = result[ (result["EnergyKWh"] > 0) & (result["DurationMinutes"] > 0)]
        result = result[result["PaymentStatus"].isin( ["Paid", "Pending", "Failed"])]
        return result

    def add_long_session_flag(self, df, duration_threshold):
        result = df.copy()
        result["IsLongSession"] = (result["DurationMinutes"] > duration_threshold).astype(int)
        return result

    def station_utilization_summary(self, df):
        result = df.groupby("StationID").agg(
            SessionCount=("SessionID", "count"),
            TotalEnergyKWh=("EnergyKWh", "sum"),
            AverageDuration=("DurationMinutes", "mean")
        ).reset_index()
        result["AverageDuration"] = result["AverageDuration"].round(1)
        return result

    def high_energy_stations(self, df, energy_threshold):
        result = df.groupby("StationID")["EnergyKWh"].sum().reset_index()
        result = result.rename(columns={"EnergyKWh": "TotalEnergyKWh"})
        result = result[result["TotalEnergyKWh"] > energy_threshold]
        return result

    def city_revenue_summary(self, df):
        result = df[df["PaymentStatus"] == "Paid"].copy()
        result["Revenue"] = result["EnergyKWh"] * 18
        result = result.groupby("City")["Revenue"].sum().reset_index()
        return result
```

Python Q9 (Numpy) — Sensor Reading Analysis System

ANSWER

Build a `SensorAnalyzer` class that creates and validates sensor arrays, computes statistics, filters extreme readings, labels high sensors, and formats readings.

Create Class and State

```
class SensorAnalyzer:
```

Step 1 — Create Sensor Array

Implementation Flow:

- Convert list to NumPy array.
- Use float dtype.
- Return array.

Function:

```
def create_sensor_array(self, sensor_values):  
    result = np.array(sensor_values, dtype=float)  
    return result
```

Step 2 — Validate Sensor Array

Implementation Flow:

- Return False if empty.
- All readings must be positive.
- Return True if valid.

Function:

```
def validate_sensor_array(self, sensor_array):  
    if sensor_array.size == 0:  
        return False  
    result = np.all(sensor_array > 0)  
    return bool(result)
```

Step 3 — Compute Sensor Statistics

Implementation Flow:

- Calculate total, average, maximum.
- Round average to 1 decimal.
- Return tuple.

Function:

```
def compute_sensor_statistics(self, sensor_array):  
    total = float(np.sum(sensor_array))  
    average = round(float(np.mean(sensor_array)), 1)  
    maximum = float(np.max(sensor_array))  
    result = (total, average, maximum)  
    return result
```

Step 4 — Filter Extreme Readings

Implementation Flow:

- Convert to float with astype.
- Readings ≥ 50.0 are reduced by 10%.
- Return updated array.

Function:

```
def filter_extreme_readings(self, sensor_array):  
    result = sensor_array.astype(float).copy()  
    mask = result >= 50.0  
    result[mask] = result[mask] * 0.9  
    return result
```

Step 5 — Label High Sensors

Implementation Flow:

- Calculate mean.
- Value $>$ mean $\rightarrow$ High.
- Otherwise $\rightarrow$ Normal.

Function:

```
def label_high_sensors(self, sensor_array):  
    mean_value = np.mean(sensor_array)  
    result = np.where(sensor_array > mean_value, "High", "Normal")  
    return result
```

Step 6 — Format Sensor Readings

Implementation Flow:

- Convert each reading to "xx.xx units".
- Return NumPy array.

Function:

```
def format_sensor_readings(self, sensor_array):  
    result = np.array([f"{x:.2f} units" for x in sensor_array])  
    return result
```

Complete Code

```
import numpy as np

class SensorAnalyzer:
    def create_sensor_array(self, sensor_values):
        result = np.array(sensor_values, dtype=float)
        return result

    def validate_sensor_array(self, sensor_array):
        if sensor_array.size == 0:
            return False
        result = np.all(sensor_array > 0)
        return bool(result)

    def compute_sensor_statistics(self, sensor_array):
        total = float(np.sum(sensor_array))
        average = round(float(np.mean(sensor_array)), 1)
        maximum = float(np.max(sensor_array))
        result = (total, average, maximum)
        return result

    def filter_extreme_readings(self, sensor_array):
        result = sensor_array.astype(float).copy()
        mask = result >= 50.0
        result[mask] = result[mask] * 0.9
        return result

    def label_high_sensors(self, sensor_array):
        mean_value = np.mean(sensor_array)
        result = np.where(sensor_array > mean_value, "High", "Normal")
        return result

    def format_sensor_readings(self, sensor_array):
        result = np.array([f"{x:.2f} units" for x in sensor_array])
        return result
```

Easy Memory Order

- Create → float array
- Validate → non-empty + positive
- Stats → sum + mean + max
- Filter → >=50 reduce 10%
- Label → > mean = High
- Format → 2 decimals + units

Python Q10 (Numpy) — Instagram Reel Trend Engagement Analysis

ANSWER

Build NumPy functions to create, validate, summarize, categorize, streak-analyze, and format daily Instagram Reel view counts.

Step 1 — Create Views Array

Implementation Flow:

- Convert list to NumPy array.
- Use dtype=np.int64.
- Return array.

Function:

```
def create_views_array(views_data):  
    result = np.array(views_data, dtype=np.int64)  
    return result
```

Step 2 — Validate Views Array

Implementation Flow:

- Return False if empty.
- Check numeric dtype.
- Check all values >= 0.

Function:

```
def validate_views_array(views_array):  
    if views_array.size == 0:  
        return False  
    if not np.issubdtype(views_array.dtype, np.number):  
        return False  
    result = np.all(views_array >= 0)  
    return bool(result)
```

Step 3 — Compute View Metrics

Implementation Flow:

- Use sum, mean, and max.
- Convert total and maximum to int.
- Round average to 2 decimals.

Function:

```
def compute_view_metrics(views_array):  
    total = int(np.sum(views_array))  
    average = round(float(np.mean(views_array)), 2)  
    maximum = int(np.max(views_array))  
    result = (total, average, maximum)  
    return result
```

Step 4 — Categorize Trend Levels

Implementation Flow:

- <3000 → "Low Trend".
- 3000–4999 → "Moderate Trend".
- >=5000 → "Viral Trend".

Function:

```
def categorize_trend_levels(views_array):
    result = np.where(
        views_array < 3000,
        "Low Trend",
        np.where(views_array < 5000, "Moderate Trend", "Viral Trend")
    )
    return result
```

Step 5 — Find Longest Growth Streak

Implementation Flow:

- Empty → 0.
- Single value → 1.
- Compare consecutive values.
- Track current and longest streak.

Function:

```
def longest_growth_streak(views_array):
    if views_array.size == 0:
        return 0
    if views_array.size == 1:
        return 1
    current = 1
    longest = 1
    for i in range(1, len(views_array)):
        if views_array[i] > views_array[i - 1]:
            current += 1
            if current > longest:
                longest = current
        else:
            current = 1
    return longest
```

Step 6 — Format View Counts

Implementation Flow:

- Convert each value to string.
- Use comma separators.
- Return NumPy array.

Function:

```
def format_view_counts(views_array):
    result = np.array([f"{int(x):,}" for x in views_array])
    return result
```

Complete Code

```
import numpy as np

def create_views_array(views_data):
    result = np.array(views_data, dtype=np.int64)
    return result

def validate_views_array(views_array):
    if views_array.size == 0:
        return False
    if not np.issubdtype(views_array.dtype, np.number):
        return False
    result = np.all(views_array >= 0)
    return bool(result)

def compute_view_metrics(views_array):
    total = int(np.sum(views_array))
    average = round(float(np.mean(views_array)), 2)
    maximum = int(np.max(views_array))
    result = (total, average, maximum)
    return result

def categorize_trend_levels(views_array):
    result = np.where(
        views_array < 3000,
        "Low Trend",
        np.where(views_array < 5000, "Moderate Trend", "Viral Trend")
    )
    return result

def longest_growth_streak(views_array):
    if views_array.size == 0:
        return 0
    if views_array.size == 1:
        return 1
    current = 1
    longest = 1
    for i in range(1, len(views_array)):
        if views_array[i] > views_array[i - 1]:
            current += 1
            if current > longest:
                longest = current
        else:
            current = 1
    return longest

def format_view_counts(views_array):
    result = np.array([f"{int(x):,}" for x in views_array])
    return result
```

Easy Memory Order

- Create → np.array int64
- Validate → non-empty + numeric + >=0
- Metrics → sum + mean + max
- Trend → <3000 / <5000 / >=5000
- Streak → consecutive increases
- Format → comma separator

Python Q11 (Numpy) — City Air Quality AQI Analysis

ANSWER

Build NumPy utilities to create, validate, summarize, categorize, and streak-analyze AQI readings.

Step 1 — Create an AQI Array

Implementation Flow:

- Convert list to NumPy array.
- Use dtype=np.int64.
- Return array.

Function:

```
def create_aqi_array(values):
    result = np.array(values, dtype=np.int64)
    return result
```

Step 2 — Validate AQI Data

Implementation Flow:

- Return False if empty.
- Check values are 0–100 inclusive.
- Return True if valid.

Function:

```
def validate_aqi_array(arr):
    if arr.size == 0:
        return False
    result = np.all((arr >= 0) & (arr <= 100))
    return bool(result)
```

Step 3 — Compute AQI Statistics

Implementation Flow:

- Calculate mean, standard deviation, maximum, and minimum.
- Round to 2 decimals.
- Return tuple.

Function:

```
def compute_aqi_stats(arr):
    average = round(float(np.mean(arr)), 2)
    std = round(float(np.std(arr)), 2)
    maximum = round(float(np.max(arr)), 2)
    minimum = round(float(np.min(arr)), 2)
    result = (average, std, maximum, minimum)
    return result
```

Step 4 — Categorize AQI Levels

Implementation Flow:

- 0–50 → "Good".
- 51–100 → "Moderate".
- 101–150 → "USG".
- 151–200 → "Unhealthy".
- 201–300 → "Very Unhealthy".
- 301–500 → "Hazardous".

Function:

```
def categorize_aqi(arr):
    result = np.where(
        arr <= 50, "Good",
        np.where(arr <= 100, "Moderate",
        np.where(arr <= 150, "USG",
        np.where(arr <= 200, "Unhealthy",
        np.where(arr <= 300, "Very Unhealthy", "Hazardous")))))
    )
    return result
```

Step 5 — Longest Unhealthy Streak

Implementation Flow:

- Unhealthy means AQI ≥ 151 .
- Track consecutive qualifying values.
- Return maximum streak.

Function:

```
def longest_unhealthy_streak(arr):
    current = 0
    longest = 0
    for value in arr:
        if value >= 151:
            current += 1
            if current > longest:
                longest = current
        else:
            current = 0
    return longest
```

Complete Code

```
import numpy as np

def create_aqi_array(values):
    result = np.array(values, dtype=np.int64)
    return result

def validate_aqi_array(arr):
    if arr.size == 0:
        return False
    result = np.all((arr >= 0) & (arr <= 100))
    return bool(result)

def compute_aqi_stats(arr):
    average = round(float(np.mean(arr)), 2)
    std = round(float(np.std(arr)), 2)
    maximum = round(float(np.max(arr)), 2)
    minimum = round(float(np.min(arr)), 2)
    result = (average, std, maximum, minimum)
    return result

def categorize_aqi(arr):
    result = np.where(
        arr <= 50, "Good",
        np.where(arr <= 100, "Moderate",
        np.where(arr <= 150, "USG",
        np.where(arr <= 200, "Unhealthy",
        np.where(arr <= 300, "Very Unhealthy", "Hazardous")))))
    )
    return result

def longest_unhealthy_streak(arr):
    current = 0
    longest = 0
    for value in arr:
        if value >= 151:
            current += 1
            if current > longest:
                longest = current
        else:
            current = 0
    return longest
```

Easy Memory Order

- Create → int64
- Validate → 0 to 100
- Stats → mean + std + max + min
- Category → nested np.where
- Streak → AQI >= 151